

# Architecture review & next-step plan

What I think. Why it matters.  
What to do next.

Keep the architecture. Make its evidence contracts enforceable.

A complete source-based review of the graph-time-series design, routing, ARBITER, SATURATE, guards, testing and release readiness. Includes 32 proposed regression cases, a staged delivery roadmap, worked examples and ten actionable work tickets.

| Preserve                                                                     | Strengthen                                                            | Prove                                                                             |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| The semantic graph, readings connection, checked interpretation and dossier. | Meaning, error states, provenance, permissions and final publication. | A defined capability set, repeatable correctness and second-building portability. |

Based on **How OntoSage Thinks**, the nine-page orientation note dated 8 September 2026. System claims are attributed to the source; recommendations and examples are explicitly identified.

9 SEPTEMBER 2026 / CORRECTED EDITION 1.1

**Accuracy revision.** Corrections are applied in place; the original pagination (pages 1-30) is retained. The correction log and audit notes are on pages 31-33. Source figures and results remain attributed, not independently validated.

HOW TO USE THIS REPORT

# Contents

Start with the decision brief, then read the source-aligned chapters. The second half turns the assessment into testable contracts, delivery gates and a concrete work backlog. Original source-page citations are kept separate from this report's page numbers.

|                                                         |       |
|---------------------------------------------------------|-------|
| Decision and evidence boundary                          | 3-4   |
| Part One - routing, meaning, time and memory            | 5-6   |
| Part Two - graph, readings and honest absence           | 7-8   |
| Part Three - orchestrator, ARBITER, SATURATE and guards | 9-12  |
| Part Four - testability and evaluation design           | 13-14 |
| Part Five - maturity and the reported scorecard         | 15    |
| Part Six - the original nine-item backlog               | 16    |
| Priority risk register                                  | 17    |
| Dependency-ordered delivery roadmap                     | 18-19 |
| 32-case regression catalogue                            | 20-21 |
| Release gates and definition of done                    | 22    |
| Worked answer and ranking examples                      | 23-24 |
| Proposed request, lane and evidence contracts           | 25-26 |
| Operational controls before a pilot                     | 27    |
| Questions that must be closed                           | 28    |
| The next ten work tickets                               | 29    |
| Sources and terminology                                 | 30    |

**Reading labels.** Source account = what the note reports. My assessment = review judgment. Proposed = a design or test recommendation. Not established = evidence is missing from the note, not necessarily from the implementation.

**Not an independent audit.** No repository, live system or benchmark was examined. The external references are limited to Brick's time-series pattern, SHACL validation and test fixtures.

## Correction log and audit notes 31-33

What changed, why it changed, and which claims still require implementation evidence.

DECISION BRIEF

# Executive decision

My recommendation is to continue with this architecture, not rewrite it. Its strongest feature is the separation between interpreting a question and establishing the facts needed to answer it. The graph-time-series connection, ARBITER’s checked CQ-IR, deterministic scoring and evidence dossier are worth preserving. They create inspectable boundaries rather than relying on a fluent answer alone. [S1, pp. 2-5, Figs. 1-4]

My confidence is higher in the architectural direction than in the completion claims. The note reports passing probes and zero open P1 bugs, alongside historical wrong-day, sensor-absence, non-returning-function, rule-property and publication incidents. Their current fix status is not independently established here. They motivate regression evidence, not a claim that all remain open or that the system needs replacing. [S1, pp. 5-7]

## What I think, and why

| Decision                                          | Reason                                                                                                          | Practical consequence                                                                                      |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Keep the graph and readings separate.             | Sensor identity and context are distinct from observations; the note gives an explicit connection between them. | Validate the connection and supported store adapters instead of changing databases without evidence.       |
| Use ARBITER as a reference pattern.               | A checked interpretation and dossier make assumptions and calculations inspectable.                             | Extend the interpretation/result contract to supported lanes, not necessarily ARBITER’s ranking algorithm. |
| Do not call the system finished on this evidence. | Test counts, routing coverage and demonstration success answer different questions from semantic correctness.   | Define a bounded capability contract and require observable release evidence.                              |
| Put honesty before feature breadth.               | A real number can answer the wrong question; an error can look like absence.                                    | Enforce time, provenance, coverage and publication checks before adding intents.                           |

## The first decision to make

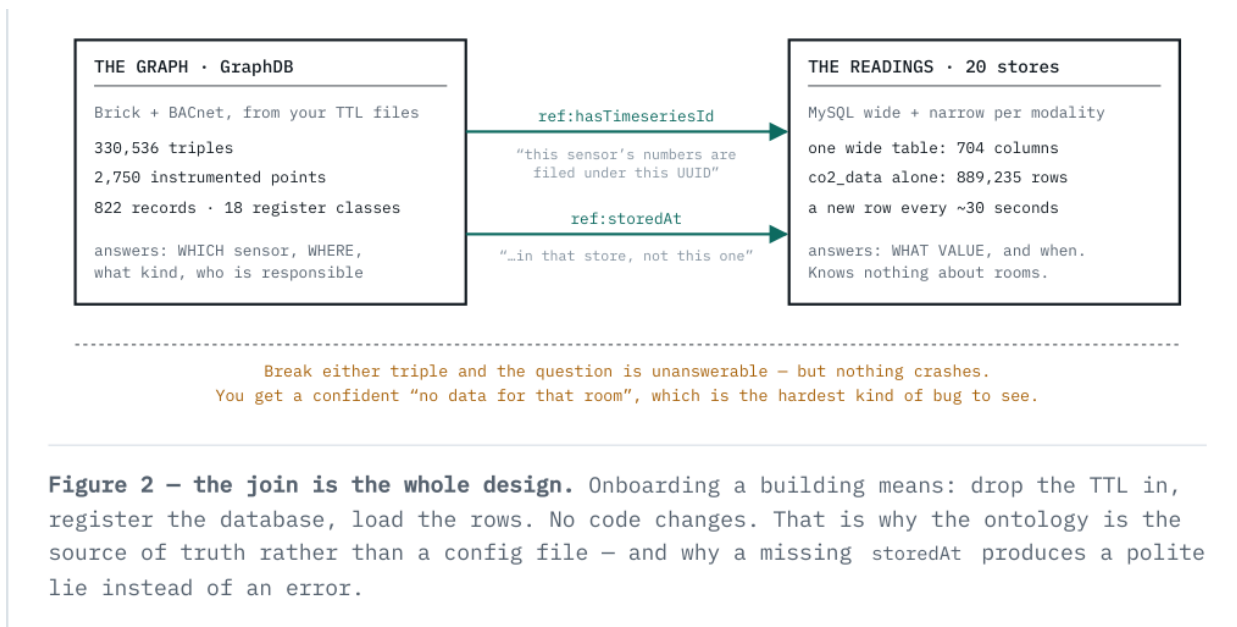
Choose the next target explicitly: **a defensible demonstration of one building, a supervised operational pilot, or portable onboarding of another building**. The source already distinguishes demonstration from self-service onboarding. This report keeps that distinction and adds separate acceptance gates; it does not treat the goals as interchangeable. [S1, pp. 6-8]

For the next release, I would prioritize a defensible demonstration with a documented supported scope. A pilot needs additional operational and privacy evidence. Portability needs a second-building test. None of those statuses can be independently certified from the orientation note alone.

The next milestone should be a defined set of building questions that OntoSage answers correctly, together with conditions under which it reliably limits or declines an answer.

# What the source reports

The source is **How OntoSage Thinks**, an orientation note dated 8 September 2026. It reports 43 TTL files, 330,536 triples, 2,750 instrumented points, 20 registered time-series backends and 822 records across 18 register classes. These are the source’s inventory figures, not measurements performed for this review. [S1, p. 1]



Reproduced source p. 3, Figure 2. Its two-reference join and no-code onboarding statement are the author's account, not verified conformance. The join is needed for relevant observation queries, not graph-only inventory.

Figure 1 on source page 2 describes follow-up rewriting, 37 intents, a 40-rule routing contract and DATA, STANDALONE and META lanes. Figure 3 on page 4 describes ARBITER’s typed CQ-IR and deterministic dossier. Figure 4 on page 5 separates referent, policy, topical grounding and numerical verification checks. Figure 5 on page 6 exposes failures across code, model and data boundaries. These figures are the basis of the six source-aligned review chapters that follow. [S1, pp. 2-6]

**Evidence boundary.** I reviewed the supplied nine-page PDF and its diagrams. I did not inspect the repository, TTL files, databases, logs, deployed policy configuration, test implementation or a running system. Consequently, statements about what is implemented remain attributed to the note. Proposed contracts, test cases, risk priorities and release gates in this report are recommendations, not descriptions of verified existing behavior.

**Notation.** [S1, p. X] always means a page of the original nine-page note. [E1]-[E3] identify narrowly scoped external implementation references on the last page. “Proposed” means a design choice to evaluate; “not established” means the note does not provide enough evidence, not that a component is necessarily absent.

# Part One: what happens to a question

## Source account

Figure 1 uses follow-up rewriting, an LLM classification into one of 37 intents, 40 deterministic rules with last-match precedence, and a lane-specific response. Documents are consulted only if no lane claims the question. The source gives a concrete reason for that ordering: a room-list query reportedly improved from six rooms mentioned in a cleaning register to 48 rooms returned from the graph. [S1, p. 2, Fig. 1]

## My assessment

**Keep explicit routing, but make precedence explainable.** A deterministic override is useful when it encodes an intentional policy. It is less reassuring when the effective policy depends on where a rule happens to sit in a file. “Last wins” can be intentional and testable; the note does not establish how conflicts are detected or documented. Forty rules are not inherently excessive. Undocumented interactions are the concern.

**Use authority by claim type, not a universal source order.** The cleaning-register example supports using the graph for the declared room inventory. It does not establish that documents should always be ignored once any other lane can answer part of a request. For example, “Which rooms had poor conditions yesterday, and what does the operating manual recommend?” requires both observations and documented guidance. This is an illustrative extension, not a reported existing failure.

## What to do next

Give each routing rule an identifier, explicit priority, intended scope and regression examples. Record the original classifier result, matched rules, final selected lane and explanation in a route trace. Reject unresolved conflicts or ask a focused question rather than relying on accidental ordering. A test should show whether rule reordering changes behavior; it should change only where precedence was deliberately specified.

Create a small claim-authority table. Declared spaces and sensor relationships can come from the graph; observations from the relevant readings; procedures from the applicable document and revision; capability answers from an inspected capability record. Record source conflicts instead of silently choosing whichever source is easiest to retrieve.

For mixed requests, begin with a **small, bounded composition rule**, not an unrestricted autonomous planner. Support a few documented combinations, preserve evidence per sub-answer, and say which part is unsupported when composition is unavailable. A lane merely claiming a question must not be treated as evidence that the entire request was answered.

## Completion condition

For every supported intent, demonstrate classification, rule selection, execution and response collection. Include overlapping rules, paraphrases, unknown rooms and mixed-source requests. A reviewer should be able to reconstruct why a route was selected and identify which authority supports each published claim. This extends, rather than discards, the source’s “one question, one road” design. [S1, p. 2]

## Part One: preserve the question's meaning

### Why this is the most important semantic boundary

The source reports a “Yesterday” report that used that afternoon’s readings and a result that treated a row limit as a count. Every retrieved figure could be genuine while the answer was still wrong. Its follow-up stage also reads the last 20 turns to resolve references such as “there.” [S1, pp. 2, 5-6]

A typed interpretation helps, but is not sufficient. A request for a maximum can be compiled into a valid request for an average. A room can exist but be the wrong referent. A date can parse correctly while selecting the wrong building-local day. These examples distinguish structural validity from semantic agreement; they do not show that every sound architecture must use this exact representation.

### Proposed normalized request

Carry one versioned request object from interpretation through rendering. It should contain the original text, building and authorized spatial identifiers, quantity, operation, aggregation basis, units, resolved time interval, time zone, requested evidence kind, assumptions, and unresolved ambiguities. Appendix-style examples appear later in this report. These fields are a proposal, not a reconstruction of the current CQ-IR.

Resolve relative dates **once**, against a recorded reference time and the building’s configured time zone. Use a documented boundary convention such as start-inclusive/end-exclusive. Convert to the storage time basis deliberately. The query, chart, table, dossier and heading should all derive their interval from that same object; the formatter should not reinterpret “yesterday” independently.

Keep the requested interval separate from observed timestamps and coverage. First and last readings do not describe internal gaps: record covered bins or segments and the coverage method. Do not call a partial interval a complete day. Treat future-looking requests separately from recorded observations; the source does not explain future hours in its afternoon example. [S1, pp. 4, 7]

### Follow-up and aggregation rules

Maintain structured conversation state for confirmed building, room, quantity and time scope. Preserve the original user wording and reference resolution. A follow-up that switches building or room should invalidate incompatible inherited context. Prior assistant assertions should not automatically become building evidence merely because they appear in the last 20 turns.

Distinguish total matching entities, retrieved rows, valid observations and displayed rows. Specify whether an average is sample-weighted, time-weighted or calculated from interval summaries. Do not silently invent an aggregation rule when it materially changes the result. Unknown units, conflicting identifiers or consequential unresolved ambiguity should trigger clarification or an explicit unsupported result.

### Completion condition

Freeze the clock in tests. Assert exact intervals, room identifiers, operations and counts in both execution and the final response. Include midnight boundaries, building-local date changes, daylight-saving transitions where applicable, conversation context switches and datasets larger than the display limit. These are proposed regression cases, not tests verified in the current suite.

STRENGTHEN THE JOIN; DO NOT REPLACE IT BLINDLY

# Part Two: the graph-readings connection

## Source account and assessment

The graph declares sensor types, locations and responsibilities; this is metadata, not proof of physical installation. The time-series stores hold observations keyed by identifiers. Figure 2 links them through `ref:hasTimeseriesId` and `ref:storedAt` and reports that a broken link can be presented as no data. [S1, pp. 2-3, Fig. 2]

This separation is coherent for questions that need both spatial meaning and observations. A declared room inventory can use the graph alone; two stores are not necessary for every question. Validate the join and adapters before considering a migration. The note supplies no comparative measurement supporting a database replacement.

Brick’s official documentation describes a `Point` linked through `ref:hasExternalReference` to a `ref:TimeseriesReference`, with a time-series identifier and database reference on that object. This supports the integration pattern, but the simplified figure is not enough to verify OntoSage’s actual TTL structure. Inspect the files before claiming schema conformance. [E1]

## Proposed connection checks

| Layer                | What to validate                                                                                                      | Failure must mean                                                |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Graph structure      | The relevant entity and relationship path exist; required reference fields have the intended types and cardinalities. | Metadata is incomplete or invalid, not “the room has no sensor.” |
| Store resolution     | The declared store maps to an authorized, configured adapter.                                                         | Unregistered store; or a separate connection failure.            |
| Series resolution    | The identifier resolves in the selected store and schema version.                                                     | Broken reference only if established; otherwise unresolved.      |
| Observation contract | Timestamp meaning, units, quantity and provenance are declared and compatible with the request.                       | The values are not yet admissible evidence.                      |
| Retrieval outcome    | Requested interval, valid rows, duplicates, coverage and backend status are recorded.                                 | A bounded result or a specific retrieval failure.                |

SHACL is one implementation option for RDF graph validation, including datatype and cardinality conditions. It does not, by itself, prove that an external database is reachable or that a referenced series returns meaningful observations. Pair graph-shape checks with active adapter checks. [E2]

## Important qualification

Do not require every declared `Point` to have time-series data unless that is part of the building’s capability contract. Apply retrieval requirements to `Points` and capabilities that actually promise stored observations. A legitimate metadata-only `Point` should not become an artificial onboarding failure. Similarly, an event-driven sensor should not be assigned a sampling cadence copied from an unrelated periodic sensor.

For supported layouts, test removed references, incorrect identifiers and unsupported units. Empty query results alone may not establish whether a series exists; record that uncertainty. After a supported mapping is repaired, rerun the same known-answer test. A genuinely new storage layout may require adapter code.

TRUTHFUL OUTCOMES AND BUILDING READINESS

# Part Two: distinguish absence from failure

The source describes a missing storedAt reference producing an apparently ordinary absence of data. It also proposes a per-building readiness report because a system can boot successfully and still answer nothing. I would move that readiness work earlier, alongside connection validation. [S1, pp. 3, 8; W3-1]

## Proposed internal outcomes

| Outcome              | Meaning                                                                      | Appropriate external behavior                                               |
|----------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| NOT_DECLARED         | No matching sensor is declared in the authorized graph scope.                | Say what the available metadata establishes; do not infer physical absence. |
| REFERENCE_MISSING    | A declared sensor lacks a required data reference.                           | Report a configuration limitation, subject to policy.                       |
| SERIES_UNRESOLVED    | Series identity not resolved; absence is not yet established.                | Do not substitute a zero, an empty-room claim or a different sensor.        |
| BACKEND_UNAVAILABLE  | Retrieval did not complete because of a store or connection failure.         | Give a temporary retrieval limitation and preserve a diagnostic internally. |
| NO_OBSERVATIONS      | Query succeeded with no rows; record series identity separately.             | Name the interval; distinguish it from inventory absence.                   |
| INSUFFICIENT_QUALITY | Data exist but are stale, invalid or too incomplete for the requested claim. | Limit or decline the affected analytical claim.                             |
| ACCESS_RESTRICTED    | Policy prevents access to the requested scope or detail.                     | Refuse or coarsen without leaking protected existence or diagnostics.       |

Outcome names are illustrative. Keep series resolution (confirmed, absent, unknown) separate from query status. If the adapter has no reliable series catalogue, an empty query cannot prove a broken identifier. Authorized diagnostics may retain distinctions that user-facing policy deliberately coarsens.

## Readiness should be capability-specific

A proposed readiness report should identify the building and graph revision; loaded capability declarations; sensor-to-store connection status; supported adapters; data freshness; known coverage gaps; enabled policy; and a small end-to-end probe result. For each capability, report ready, degraded, unsupported or failed, with a concrete reason and remediation owner.

A building can legitimately be ready for room inventory and not ready for CO2 analytics. A backend health check is not an answer-quality check. Conversely, a missing optional modality should not make every unrelated capability red. Readiness must answer **what can this building currently support, for which scope, and with what limitations?**

## Completion condition

Seed each outcome deliberately and assert the corresponding internal result and authorized external wording. Verify that neither an exception wrapper nor response collection converts a failure into ordinary success. Demonstrate that an operator can identify a broken reference from the readiness report without discovering it through a misleading end-user answer.

The central principle is narrow but important: **absence in a graph, absence in a time window, failed retrieval and forbidden access are different facts**. Keeping them separate is part of factual correctness, not merely operational convenience.

## Part Three: the orchestrator and LLM boundary

### Source account

The note describes a LangGraph state machine that loads intents from YAML, registers nodes, wraps lane failures and shares state with reserved keys. It also reports a lane that routed and computed correctly but whose answer was not collected for the final response. [S1, pp. 3-4]

There is an unresolved inconsistency. Part One says classification is the only LLM step and later execution is deterministic. Figure 5 lists the model as classifying, writing SQL, writing Python and narrating; the backlog still includes SPARQL examples in six prompts. These could describe different lanes or versions, but the note does not say which. [S1, pp. 1-2, 6, 8]

### What I would require before making stronger claims

Create an **LLM call-site inventory** with lane, call site, input, output type, model/prompt version, validation, execution permissions and fallback. Include follow-up rewriting, classification, CQ-IR compilation, any SQL/SPARQL/Python generation, document synthesis, verification and final narration. Mark each as confirmed, absent or still to inspect. Do not assume that a deterministic executor makes its generated input semantically correct.

For supported analytical operations, I would prefer closed-vocabulary interpretation followed by validated query builders and fixed computation routines. Where generated queries or code remain necessary, document that broader boundary honestly and impose a separate execution policy. That is a conditional recommendation; the PDF is not enough to establish which paths currently execute generated code.

### Lane and state contracts

Inspect the existing lane-result contract and strengthen it where needed: status, structured answer data, evidence, scope, warnings and failures. The note describes reserved state keys but does not provide their schemas. Test ownership and unexpected overwrites, and define outcomes for None returns, late completion and partial evidence. [S1, p. 3]

Generate or validate registration, routing and response collection from the same intent manifest. The manifest should describe the handler, supported input schema, result schema and response adapter. Add a startup check and an end-to-end contract test for each declared intent. This directly addresses the missing collection step reported in the note without requiring “graph surgery.” [S1, pp. 3-4]

### Completion condition

Every LLM output has a named consumer and validation boundary. Every lane either produces a well-typed result or an explicit failure. A newly registered intent cannot pass acceptance without a rendered answer test. One failing lane must not erase independently valid results, but partial completion must remain visible.

**The documentation should describe the implementation that actually runs.** Until the inventory resolves the inconsistency, present “one neural step, then arithmetic” as the stated ARBITER design, not as a proven system-wide property. [S1, p. 4, Fig. 3]

## Part Three: ARBITER

### Source account and what I would keep

ARBITER compiles a question into a checked, closed-vocabulary CQ-IR. It then admits the request or asks one question, enumerates spaces, records coverage, fetches and scores evidence, and emits a dossier. The source says that ranking and comparison bands come from code, and the dossier records interpretation, assumptions and a candidate-by-criterion evidence table. [S1, p. 4, Fig. 3]

**This is the strongest part of the design.** It makes the reasoning contract inspectable: which candidates were considered, which criteria were used and what evidence supports the figures. I would reuse that contract pattern across supported analytics, while keeping ARBITER's actual ranking logic specific to ranking tasks.

### Why deterministic scoring is not enough

A reproducible score can still encode the wrong objective. "Best for focused work" may involve temperature, noise, air quality, availability or user-specific constraints. The note does not identify the exact criterion definitions, aggregation rules, weights, standards and versions, or how missing values affect eligibility. These are not proven defects; they are necessary details to inspect before trusting an operational recommendation. [S1, pp. 4, 7]

### Proposed ranking contract

Separate **hard constraints** from **preferences**. Authorization, requested location, required availability and required evidence coverage should not be traded away for a higher comfort score. Apply those eligibility conditions before ranking. Then record criterion direction, units, transformation into a comparable score, weight, aggregation interval and applicable threshold reference.

Store the exact standard, edition and context used for each comparison band. Do not label a result compliant, healthy or safe merely because one code threshold was passed. Any such label needs a clearly bounded definition and the appropriate authority. No specific environmental threshold is endorsed by this report.

Treat missing evidence explicitly. Do not automatically reward a room because an unfavorable criterion is missing. Use a declared common-coverage rule, a clearly labeled partial ranking or an abstention. Report ties and sensitivity to reasonable weight changes; a tiny score difference should not be narrated as an unequivocal winner.

### Clarification and time

A one-question budget is a product constraint, not permission to proceed through a consequential unresolved ambiguity. Ask the most decision-relevant question, then either use declared acceptable defaults, narrow the answer or stop. The user should see when the conclusion depends on an assumption. [S1, p. 4]

A request involving future hours needs forecasts or a statement that the ranking uses currently observed conditions only. The note's live-reading example does not establish future availability or future environmental performance. [S1, p. 7]

**Done means** a reviewer can reproduce the score from admissible evidence and explain why the recommended room is eligible, which alternatives were excluded and how robust the ordering is under the declared criteria.

MAKE SIMULATION USEFUL WITHOUT MAKING IT LOOK OBSERVED

# Part Three: SATURATE

## Source account

SATURATE fills missing space-by-quantity combinations with simulated sensors that carry graph metadata and time-series references. The note says the gaps are filled visibly rather than hidden. That is a valuable intention, but it does not establish exactly how simulated evidence is governed in operational answers. [S1, p. 4]

## My assessment

**Keep SATURATE for testing, demonstrations and explicit simulation. Do not let structural completeness stand in for observational evidence.** A simulated Point can satisfy the same graph and storage contracts as a physical Point. That makes it useful for exercising the pipeline, but it does not show that its values describe the actual room.

The risk is specific: a fully traceable, deterministic ranking could still be dominated by generated evidence. A provenance check that only asks “does this number exist in a store?” would not establish the provenance needed for a real-building claim. This is a design risk inferred from the described mechanism, not an allegation that such an answer has already been observed.

## Proposed evidence modes

| Mode                    | Default admissible evidence                                                | Required presentation                                                                          |
|-------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Operational observation | Measurements and approved deterministic summaries of them.                 | Observation interval, freshness, coverage and exclusions.                                      |
| Explicit estimation     | Approved estimation method and declared input lineage.                     | Method, input lineage, validation limits and uncertainty where supported.                      |
| Explicit simulation     | Simulated data in a named scenario, optionally compared with observations. | A prominent scenario label and a clear statement that it is not measured building performance. |

Track evidence origin separately from transformation and claim type. A maximum, converted unit or preference score is computed, not a raw measurement. Keep parent lineage through each transform, including mixed origins. Permit approved summaries of measurement-origin inputs; require separate admission for estimates and simulations. A single measured/derived/simulated enum is insufficient.

## Enforcement choices

My proposed default excludes simulation-origin and unapproved estimate-origin inputs from operational observation claims, including their downstream summaries. Retain measurement, estimation and simulation coverage separately. Explicit scenario mode may admit simulations with labeling. Apply provenance rules per observation or batch, not only per Point; stores or namespaces alone do not enforce them.

If simulation is used to evaluate the system, record the generator version, scenario assumptions and seed where applicable. Use simulations to test known behavior, not to claim validation against an unobserved real building. Check that generated “complete” coverage has not hidden the very absence states the application must handle correctly.

## Completion condition

Under this proposed policy, an attractive simulated candidate must not win an operational ranking, even after its values are transformed. Approved unit conversions and summaries of measured samples remain admissible. Simulated coverage must not inflate measured coverage. Preserve these distinctions in charts and exported reports as well as on screen.

A FAILED CHECK MUST CHANGE WHAT THE USER RECEIVES

# Part Three: guards and evidence publication

## Source account

Figure 4 separates a referent gate, policy engine, topical grounding guard and numerical verifier. It places a single evidence record after the lanes. The same page reports a result marked grounded=False that was nevertheless sent, plus wrong-window and row-limit errors that passed existing checks. [S1, p. 5, Fig. 4]

## My assessment

The separate checks are sensible, but a boolean annotation is not a publication policy. Keep the central evidence record and check numerical lineage separately from agreement with the normalized request. That agreement still does not prove the initial interpretation was right. A failed verifier means the required check did not pass; it does not by itself prove the underlying calculation false.

## Proposed publication contract

| Check                  | Question it must answer                                                                | On failure                                                |
|------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Referent and scope     | Are the building, rooms and entities the authorized requested ones?                    | Clarify, limit or refuse the affected claim.              |
| Policy                 | Is this disclosure permitted at this resolution, including derived output?             | Suppress or coarsen; do not reveal protected diagnostics. |
| Semantics              | Do quantity, operation, units and time interval match the request?                     | Recompute through a bounded path or withhold.             |
| Provenance and quality | Are origin, transforms and task-specific quality admissible?                           | Label a permitted partial result or decline it.           |
| Numerical lineage      | Can each quantitative claim be reproduced from its recorded inputs and transformation? | Remove the unsupported claim.                             |
| Rendering integrity    | Do the text, table and chart express the same verified result?                         | Block inconsistent output.                                |

For partial answers, publish only claims whose dependencies passed. Then recheck the combined response: a remaining phrase such as “best overall” might still depend on an excluded criterion. A policy failure must be respected even when unrelated arithmetic succeeds.

## Evidence worth retaining

Record request and interpretation identifiers, route decisions, graph and policy versions, resolved time boundaries, authorized scope, selected sensors and stores, retrieval status, transformations, units, provenance, coverage, claim-to-evidence references, verification decisions and omissions. Keep sensitive raw data and credentials out of ordinary logs. The retention and access rules themselves need an owner.

The policy gate should also constrain retrieval and derived outputs, not merely decorate the answer afterward. The source describes privacy refusal and coarsening but does not establish enforcement at every stage; this is an inspection requirement, not a verified deficiency. [S1, pp. 2, 5]

**Done means** a failed evidence check demonstrably changes the final text, chart and export. An adversarially altered date, unit, count or provenance label should be rejected even when all displayed numbers occur somewhere in the database.

THE REPORTED DEFECTS ARE TESTABLE

# Part Four: why the list keeps growing

## The source’s explanation

Part Four says the serious defects live between the model, graph and database, beyond the reach of the 5,419 automated tests. It presents careful end-to-end questioning as the way those defects were found. The later work plan already proposes converting seven earlier failures into permanent test questions. [S1, pp. 5-6, 8; W4-4]

**I agree with the need for end-to-end observation. I disagree that these defects are inherently untestable.** Discovery and regression prevention are different tasks. A person may discover the failure by using the system; a controlled dataset and explicit expected result can then turn that discovery into a repeatable test. The absence of such a test in the current suite does not make it impossible to write.

## Turn each reported defect into an observable contract

| Reported defect                           | Concrete test design                                                                          | What must be asserted                                                                                   |
|-------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Sensor-list function has no return.       | Provide a graph fixture with known matching sensors, and call the actual function.            | Exact return type and identifiers; unexpected None fails rather than becoming an empty set.             |
| Sensor presence was described as absence. | Provide a declared sensor with a resolvable series and known rows; exercise the full request. | Sensor presence is recognized and no installation recommendation is generated from a retrieval mistake. |
| “Yesterday” used that afternoon.          | Freeze the reference clock and building time zone; seed distinct values on adjacent days.     | Query boundaries, result values and output heading all use the requested day.                           |
| Row limit became a count.                 | Seed more matching entities or observations than the fetch/display limit.                     | Total, retrieved and displayed counts remain distinct.                                                  |
| ECA rules use the wrong property.         | Seed one rule and the intended graph relationship with a known triggering event.              | The expected rule matches; a non-matching fixture does not.                                             |
| Ungrounded report was published.          | Force the verification result to fail and exercise final rendering/export.                    | Unsupported claims do not reach the user or an exported report.                                         |

Incident basis: source pp. 4-6. These are proposed test designs; they have not been executed against OntoSage for this review.

Fixtures can provide a known dataset or configured database; pytest’s own documentation explicitly describes those roles. That is sufficient support for the testing mechanism, not evidence that OntoSage already uses it correctly. [E3]

## Change the working rule

For each discovered defect, retain a reproducer, expected outcome, regression test and repair record. If already fixed, use the historical revision or a controlled mutation in an isolated test environment to confirm detection; never reintroduce a defect into a live system. Continue live exploration for new failure classes rather than relying on rediscovery of known ones.

These incidents show that the checks exercised at the time did not prevent the reported outcomes. They are testable contract failures, not inherently unreachable seams. The note alone cannot show which regressions have since been added, which fixes are current, or what coverage is still missing. Verify those points before sizing repair work.

TEST MEANING, EXECUTION AND FINAL OUTPUT SEPARATELY

# Part Four: build an evaluation system

## Proposed evaluation layers

Retain the existing suite, but organize evidence by the property it establishes. A large count of unit tests is useful; it cannot substitute for a small number of well-chosen end-to-end tests. The source’s reported live probe and demo questions are additional evidence, not interchangeable measures. [S1, pp. 6-7]

| Layer                | Controlled input                                              | Primary assertion                                                                       |
|----------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Pure computation     | Fixed values, units and aggregation parameters.               | Correct transformations, scores, ties and counts.                                       |
| Graph/store contract | A small known graph and each supported adapter layout.        | Correct sensor discovery, reference resolution and failure classification.              |
| Interpretation       | Original questions and expected normalized requests.          | Correct entity, quantity, operation and time meaning.                                   |
| Lane integration     | A normalized request through the actual lane.                 | Structured result, evidence and errors are complete.                                    |
| End-to-end response  | User wording through interpretation, retrieval and rendering. | The final answer and chart satisfy the requested meaning.                               |
| Live monitoring      | A bounded probe against the current deployment.               | Operational availability and freshness, without replacing controlled correctness tests. |

## Keep three datasets distinct

Use a **development regression set** for known failures and supported behavior; a **held-out evaluation set** for requests not used to tune routing or prompts; and a **live probe set** for deployment checks. Version all three. When an evaluation case is used to tune the implementation, move it into development and replenish the held-out set rather than continuing to describe it as unseen.

Build a small reference building with known room identifiers, overlapping labels, multiple sensor types, supported store formats and deliberately varied time-series values. Include missing references, no observations, stale data, access restrictions and simulated values. A fixture should make the expected answer independently calculable rather than merely resemble production data.

## Assess the two uncertain boundaries directly

Score interpretation separately from execution. A failed end-to-end answer should be attributable to the interpreted request, selected entities, retrieval, calculation, policy, verification or rendering. Compare structured outputs first; then check that the final language preserves them. Do not use a model’s fluency judgment as the sole oracle for numerical or temporal correctness.

Repeat relevant model-dependent cases and keep the model, prompt, graph, adapter and policy versions in the result. Record the first attempt, retries and final outcome separately. The source’s earlier 49/50 runs that passed on retry should remain visible as first-pass failures, not disappear inside a success total. [S1, pp. 8-9]

## Completion condition

Each supported capability has a known-answer case, boundary cases and an expected inability case. Each reported defect has a regression reproducer. Held-out results identify both wrong answers and unnecessary abstentions. A failed run leaves enough structured evidence to locate the responsible boundary without guessing from the final paragraph alone.

INTERPRET THE REPORTED SCORECARD WITHOUT OVERCLAIMING

# Part Five: where OntoSage actually is

## What the note reports

The scorecard on source page 7 lists 5,419 offline tests, a 50/50 live probe, 10/10 demo questions, a typical answer time of 14 seconds, zero open P1 bugs and zero hardcoded literals. It also says all 37 intents route and the listed building capabilities are demonstrable. These are reported results; this review did not reproduce them. [S1, p. 7]

| Reported item              | Reasonable interpretation                                         | What it does not establish                                                            |
|----------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 5,419 offline tests        | There is a substantial reported automated suite.                  | The tested properties, fixture realism or coverage of the documented failure classes. |
| 50/50 live probe           | One reported set of 50 questions passed its chosen assessment.    | General accuracy, independence from tuning, repeatability or the pass rubric.         |
| 10/10 demo questions       | The selected demonstration reportedly works.                      | A broad capability boundary or adversarial robustness.                                |
| Typical answer: 14 seconds | The note gives a useful indicative latency.                       | Median, tail latency, concurrency, timeout rate or cost per successful answer.        |
| Zero open P1 bugs          | No bugs in that reported severity category remain open.           | No unknown defects or no unsupported claims.                                          |
| Zero hardcoded literals    | The note reports zero under an unspecified definition and method. | Freedom from building-specific vocabulary, prompt examples or adapter assumptions.    |

The last point needs particular care: the backlog still includes graph-derived routing vocabulary and six prompts containing building-specific room examples. These statements may refer to different definitions of “hardcoded,” but the note does not resolve that distinction. [S1, p. 8; W2-1, W2-5]

## My maturity assessment

**Conceptual direction: credible. Implementation breadth: substantial according to the source. Reliability and portability: not independently established.** This is not a claim that the system is unusable. It is a distinction between an encouraging orientation note and the evidence needed for a release decision.

I would replace “the idea is finished” as a project-management measure with a capability matrix: supported question type, required building data, known limitations, evaluation coverage and release status. A feature can be implemented but not sufficiently evaluated; another can be safely supported only with a stated limitation.

## Better operational reporting

Report substantive-answer correctness, completion on supported and answerable requests, justified refusals, unnecessary abstentions, incorrect assertions, policy failures and operational failures separately. Add workload-specific latency percentiles and relevant data freshness. Publish denominators, scope and retry outcomes with every rate.

A perfect finite test run is evidence about that run, not proof of universal reliability. The appropriate next claim is therefore narrower: **this version passed these specified cases under these recorded conditions, and these unsupported situations are handled explicitly.** That claim is both useful and auditable.

PRESERVE THE SOURCE BACKLOG; REORDER BY DEPENDENCY

# Part Six: what is genuinely left

The note selects six of nine items. Their estimates total 7 days; the three deferred items total 9, so the visible table totals 16, not the stated 19. The retained .env.example subset adds about 0.5 day if not already included. The source does not explain the discrepancy. These are source estimates and arithmetic, not validated delivery durations. [S1, p. 8]

| Source item | Source purpose / estimate                                                | My recommendation                                                                                                                                                                    |
|-------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| W4-4        | Seven earlier failures become permanent test questions; 1 day.           | Start immediately. Obtain the exact seven-case list; the PDF does not enumerate it fully. Add final-output assertions, not only route checks.                                        |
| W2-1        | Routing vocabulary comes from the building graph; 1 day.                 | Keep. First freeze regression cases and define ambiguous-name behavior. Test with another building's labels.                                                                         |
| W2-5        | Generate SPARQL examples from the live graph; 1 day.                     | Keep conditionally. First inspect which lanes actually generate queries and how examples are validated. Remove building-specific assumptions without expanding execution privileges. |
| W3-4        | Close three declared record types that answer nothing; 1 day.            | Keep for types in the release scope. Give unsupported types an honest capability status rather than an apparently successful empty answer.                                           |
| W3-1        | Per-building readiness report; 1 day.                                    | Move earlier, alongside graph/store validation. Readiness should be capability-specific and reveal broken connections.                                                               |
| W4-1        | Test and document the deliberation lane; 2 days.                         | Keep and expand acceptance criteria to coverage, provenance, weighting, time interpretation and publication.                                                                         |
| W3-2        | Register scaffolder; 3 days.                                             | Defer for a single-building demo. Reconsider after manual second-building onboarding identifies repeated work.                                                                       |
| W3-3        | Templates for 24 TTL files; 3 days.                                      | Defer generic template production until the required building contract is stable.                                                                                                    |
| W3-5        | Deployment / naming: 3 days; retain .env.example subset (about 0.5 day). | Defer cosmetic/generalized packaging, not essential reproducibility, secrets handling or the ability to run the chosen release safely.                                               |

Item names and estimates above are reported from source p. 8. Recommendation scope and ordering are this review's proposals.

## Add the missing cross-cutting acceptance work

Before closing the selected backlog, verify coverage of request consistency, explicit error outcomes, publication enforcement and simulation governance. Some controls or repairs may already exist or fit within the source items. Inspect first; do not automatically add a parallel framework or treat a historical defect as still open.

The source's six selected items are a useful starting scope, not a complete proof of operational readiness. A sensible execution order is: reproduce known failures; inspect LLM and lane contracts; contain unsupported publication and simulation; validate connections and request semantics; then finish vocabulary, record types and ARBITER evaluation.

**Do not measure progress by the length of the todo list.** Measure whether the supported capability set is more clearly defined, known failure modes remain fixed and unsupported requests produce honest outcomes. That retains the source's constructive framing while replacing an open-ended cleanup story with explicit exit conditions. [S1, pp. 8-9]

## PROPOSED TRIAGE, NOT A MEASUREMENT OF INCIDENT FREQUENCY

# Priority risk register

Priorities are review proposals, not the source's P1 labels or measured probabilities. Reported incidents are historical; current fix status is unverified. The source separately reports zero open P1 bugs. Entries marked to inspect are possible risks, not diagnosed failures. Confirm controls before creating or reopening work. [S1, pp. 5-7]

| ID / priority                            | Risk and evidence status                                                                                        | Why it matters                                                    | Required control                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------|
| R1 / release gate                        | Failed-verification claim was published. Historical. [S1, p. 5]                                                 | Users receive a claim the system itself could not support.        | Enforced claim-level publication decision, including charts and exports. |
| R2 / release gate                        | Correct numbers answer the wrong date, entity, operation or count. Date/count incidents reported. [S1, pp. 5-6] | Numerical provenance can conceal semantic error.                  | One normalized request and end-to-end semantic assertions.               |
| R3 / release gate                        | Retrieval failure becomes sensor or data absence. Reported. [S1, pp. 3, 5-6]                                    | Misdiagnosis can lead to unjustified operational recommendations. | Typed outcomes, connection validation and readiness checks.              |
| R4 / release gate when enabled           | Simulated evidence influences operational claims without effective separation. To inspect. [S1, p. 4]           | Structural validity does not establish measured conditions.       | Provenance-aware admissibility and explicit simulation mode.             |
| R5 / release gate for private data       | Possible policy gaps in retrieval, derivatives or caches, if used. To inspect. [S1, p. 5]                       | A correct answer can still be an unauthorized disclosure.         | End-to-end authorization and disclosure tests.                           |
| R6 / high                                | LLM boundary is described inconsistently. Actual scope unverified. [S1, pp. 1-2, 6, 8]                          | Reliability and execution assumptions may be wrong.               | Call-site inventory, validated outputs and scoped execution.             |
| R7 / high                                | A lane computes but its answer is lost. Reported. [S1, p. 4]                                                    | Route coverage overstates usable capability.                      | Manifest-to-handler-to-renderer contract tests.                          |
| R8 / high                                | Possible objective or coverage weakness in ranking. To inspect. [S1, p. 4]                                      | "Best" can overstate what the evidence establishes.               | Eligibility, criterion and coverage contracts with sensitivity checks.   |
| R9 / later unless portability is claimed | Another building needs code or hidden prompt changes. To inspect. [S1, pp. 3, 8]                                | An onboarding claim may not transfer beyond the first building.   | Second-building trial with a recorded intervention log.                  |

## How to use this register

Assign each in-scope risk an accountable owner, a reproducer or inspection task, a control, an acceptance test and closure evidence. One person can hold several roles. Do not close a risk because a guard was added; close it when the relevant bad outcome is prevented in an end-to-end test.

For a demo, restrict or disable capabilities whose release-gate risks remain unresolved. That is preferable to either hiding limitations or delaying every supported feature until every future product risk has been addressed. Reopen a closed risk when its model, policy, data mapping or execution boundary changes materially.

# Delivery roadmap: contain and formalize

This roadmap is a proposed sequence, not a promise of a particular number of days. Re-estimate after Phase 0 using the actual code and reproducible failures. Tests are written alongside each change; evaluation is not postponed until the final phase.

## Phase 0 - establish the release boundary

**Work.** Select the release target and preserve the current revision, configuration and probe set. Verify the present status of historical incidents before planning repairs. Inventory LLM calls, lane contracts, evidence and guards. Obtain W4-4's exact seven cases, W3-4's three record types and the scorecard definitions. [S1, pp. 7-8]

**Deliverables.** A capability matrix, architecture/call-site map, reproducible baseline and short decision log. Record what is verified in code, what is merely documented and what remains unknown.

**Exit.** Every supported lane has a known execution path, response owner and acceptance case. Unsupported capabilities are identified. The team knows whether generated SQL, SPARQL, Python or narrative exists on each path.

## Phase 1 - prevent misleading success

**Work.** Verify publication enforcement, explicit failure states, partial-answer limitations and simulation policy; repair only confirmed gaps. Include in-scope permissions and response collection. Preserve working controls and their tests rather than reimplementing them solely because the note describes an earlier incident.

**Deliverables.** A typed lane-result contract, initial publication policy, provenance admissibility rules and tests for R1, R3, R4, R5 and R7. Where a complete repair is larger, temporarily disable the affected claim type rather than presenting it as supported.

**Exit.** Forced verifier failure cannot publish the affected claim. Broken references are not rendered as sensor absence. Simulated values cannot enter operational claims unnoticed. A correctly executed supported lane produces the expected response or an explicit failure.

## Phase 2 - formalize meaning and data connections

**Work.** Introduce or strengthen the normalized request object. Resolve time once; distinguish counts from display limits; validate units and aggregation. Validate graph shapes and live references for supported capabilities. Bring the readiness report forward. Then replace building-specific routing vocabulary and prompt examples behind a regression safety net.

**Deliverables.** Request and connection contracts, graph/adaptor checks, a capability readiness report, route traces and controlled fixtures for each supported storage layout. Relevant source items are W3-1, W2-1 and W2-5, with W4-4 protecting the changes. [S1, p. 8]

**Exit.** Wrong-day, wrong-operation, row-limit, ambiguous-room and broken-reference fixtures fail safely or answer correctly. The final text and chart use the same resolved request as execution. Readiness identifies failures before users discover them through misleading answers.

# Delivery roadmap: prove and transfer

## Phase 3 - establish a defensible evaluated release

**Work.** Complete in-scope record types and ARBITER’s eligibility, criterion, coverage and provenance contracts. Exercise the regression catalogue in this report. Add a held-out question set, adversarial cases and mixed-source requests that the release explicitly supports. Evaluate interpretation and final answers separately. Repeat model-dependent runs and preserve first-pass failures and retry outcomes.

**Deliverables.** Versioned test fixtures, per-capability results, an ARBITER dossier example, a limitation register, release notes and a demonstration script that shows both successful answers and justified inability. Source items W3-4 and W4-1 belong here, though their regression work starts earlier. [S1, p. 8]

**Exit.** The scoped release passes its documented hard gates; every known serious incident in scope has a regression test; and an independent reviewer can reproduce representative answers from the dossier. Success on a fixed set is reported as such, not promoted into a universal accuracy claim.

## Phase 4 - test portability before automating it

**Work.** Select a second building with different naming conventions, missing modalities and at least one different supported storage layout. Use the same application revision. Record every change to metadata, mappings, adapter configuration, prompts and application code. Repeat known-answer tests and the readiness process.

**Deliverables.** A second-building onboarding log, capability comparison and list of repeated manual tasks. Build scaffolding or templates only where that log demonstrates a recurring need. If the second building requires a new adapter, separate that legitimate scope expansion from failure to support an already declared layout.

**Exit.** A no-application-code-change onboarding claim is limited to the tested building contract and adapter set. Unexpected code or prompt changes are explained. Broader self-service claims wait until another person can follow the documented process successfully.

## Accountable roles

| Role                       | Owns the decision or evidence                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------|
| Product / system owner     | Supported capability scope, acceptable assumptions, user-facing limitations and release decision. |
| Application engineer       | Request, route, lane, computation and publication contracts.                                      |
| Data / ontology owner      | Graph validity, identity mappings, provenance, units, freshness and readiness.                    |
| Evaluator / reviewer       | Known-answer fixtures, held-out cases, severity classification and closure evidence.              |
| Operations / privacy owner | Permissions, restricted output, observability, retention and rollback.                            |

These are responsibilities, not a required team size. A solo builder can use the roles as separate review passes. A second reviewer is especially useful for expected answers and ranking criteria that the implementer otherwise grades against their own assumptions.

**Working rhythm.** For each defect: capture evidence, reduce it to a test, repair, rerun the relevant scope and record closure. At each milestone, review capability coverage and unresolved release gates. Do not use raw test count or raw backlog length as the main progress measure.

# Regression catalogue A

Use deterministic fixtures and exact expected structured outputs wherever possible. These cases are proposed specifications, not executed results. Some derive from reported incidents; others are boundary cases added by this review. The original seven-case W4-4 list still needs to be obtained. [S1, pp. 5-6, 8]

| ID  | Setup or request                                                                | Required outcome                                                                          |
|-----|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| A01 | Known graph with two matching sensors; call the sensor-list function.           | Exact identifiers and declared return type; unexpected None fails loudly internally.      |
| A02 | Declared sensor has a working store reference and seeded readings.              | Presence and data are recognized; no claim that installation is required.                 |
| A03 | Remove only the required store reference.                                       | REFERENCE_MISSING or equivalent; no physical-absence inference.                           |
| A04 | Use a store identifier that is not registered.                                  | Specific store-resolution failure; no substitution with a different backend.              |
| A05 | Wrong series ID; test adapters with and without an existence catalogue.         | Establish missing series via catalogue, or report unknown resolution; no false certainty. |
| A06 | Valid series has no observations in the requested interval.                     | Empty-window result; series existence is independently confirmed or left unknown.         |
| A07 | Seed yesterday and today with intentionally different values; freeze the clock. | Exact building-local requested day appears in query, result and heading.                  |
| A08 | Run around midnight and a daylight-saving transition where relevant.            | Boundaries follow the configured time-zone policy, not a hardcoded 24-hour subtraction.   |
| A09 | More matching records than the fetch or display limit.                          | Known total is independent of truncation; unknown totals are not invented.                |
| A10 | Ask for maximum, minimum and average over the same data.                        | Each request uses its intended operation and aggregation contract.                        |
| A11 | Supply compatible convertible units, then unknown or incompatible units.        | Approved conversion is explicit; unsupported combinations are rejected.                   |
| A12 | Add duplicate rows, invalid values and out-of-order timestamps.                 | Declared quality policy applies; accepted and excluded counts are recorded.               |
| A13 | Two rooms share a label across buildings or floors.                             | Authorized identity is resolved or clarified; no silent cross-building selection.         |
| A14 | Follow-up changes room or building after a prior answer.                        | Incompatible inherited state is cleared; references resolve to the intended context.      |
| A15 | Retrieve equivalent seeded data through each supported wide/narrow adapter.     | Equivalent semantic results and evidence despite storage-layout differences.              |
| A16 | Change a mapping or graph revision between discovery and retrieval.             | The defined version/snapshot policy prevents an unexplained mixed result.                 |

## Test oracle and storage discipline

For A07 and A10, calculate expected values independently of the production query builder. For A09, verify the real total using the fixture's known contents, not by asking the same limited retrieval path to grade itself. For A15, use each actual supported adapter against a controlled compatible store; a mock alone does not establish adapter compatibility.

Retain the original request, normalized request, selected entity IDs, store/series IDs, time bounds, values and final answer with each failure. This creates a useful diagnostic record without requiring a person to reconstruct the entire pipeline from prose.

# Regression catalogue B

The second group targets response collection, rule execution, ARBITER, SATURATE and the guards described in the note. It also adds conditional operational checks where the implementation boundary remains unknown. [S1, pp. 2-6]

| ID  | Setup or request                                                                           | Required outcome                                                                            |
|-----|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| B01 | Two routing rules match the same supported request.                                        | Documented priority resolves the conflict; the route trace records why.                     |
| B02 | A supported intent runs successfully but its response adapter is removed.                  | Startup or contract validation fails; no generic success-without-answer state.              |
| B03 | An ECA fixture contains the intended relationship and triggering event.                    | Expected rule match is observed; negative fixture does not match. No actuation is required. |
| B04 | Force numerical verification failure.                                                      | Affected claims are withheld in text, charts and exported reports.                          |
| B05 | Keep numbers unchanged but alter the rendered day, unit or room.                           | Semantic/rendering verification blocks the inconsistent answer.                             |
| B06 | Simulated candidate scores best; also transform its data and measured data.                | Exclude simulated lineage in operational mode; allow approved measured-data summaries.      |
| B07 | Remove a required criterion from one candidate.                                            | Eligibility or partial-ranking policy applies; missing evidence is not rewarded.            |
| B08 | Change weights slightly; include exact and near ties.                                      | Ranking follows the declared policy; tie/sensitivity limitations remain visible.            |
| B09 | Request future conditions with only observations available.                                | Current-data limitation or unsupported forecast result; no invented future certainty.       |
| B10 | Clarification is exhausted but a consequential ambiguity remains.                          | Narrowed answer or abstention rather than silent guessing.                                  |
| B11 | Unauthorized room, restricted occupancy detail or derived sensitive output.                | Policy applies before disclosure, including dossier and chart content.                      |
| B12 | If a cache exists, change permissions or building scope before reusing an answer.          | Cache cannot bypass the current authorization and provenance policy.                        |
| B13 | Document shares keywords but is irrelevant; another contradicts a current fact.            | Irrelevant material is rejected; conflict is surfaced under the authority policy.           |
| B14 | Ask a supported observations-plus-procedure question.                                      | Both subclaims have their own sources, or the unsupported part is stated.                   |
| B15 | Inject a lane timeout, backend outage and delayed completion.                              | Explicit partial/failure outcome; retry and first-pass status remain distinguishable.       |
| B16 | Where generated queries/code exist, supply instructions that request disallowed execution. | Validated permissions and execution limits hold; untrusted text cannot grant privileges.    |

## Stronger checks than “a response exists”

For B04 and B05, inspect the actual final response artifacts, not only the verifier flag. For B06 and B07, assert which candidate is eligible and why, not just whether the score is numeric. For B11 and B12, inspect every output channel and test with identities whose permitted scopes differ.

B16 is conditional: the source does not establish whether generated code is executed on every path. The LLM inventory determines where this test is required. If an entire execution capability is unnecessary for the release, disabling it is a legitimate control, provided the documented supported scope is updated.

## DIFFERENT CLAIMS REQUIRE DIFFERENT EVIDENCE

# Release gates and definition of done

These are proposed acceptance criteria. They are not results already achieved, a safety certification or a promise that finite testing eliminates every unknown defect. The owner should approve them before the final evaluation rather than changing the bar after seeing results.

### Gate A - defensible demonstration

Document the supported building, questions, versions and limitations. Pass applicable hard-gate regressions and demonstrate both answers and justified inability. Every quantitative claim needs reproducible evidence with matching scope, units, interval and origin; rankings use ARBITER's dossier. Keep simulated scenarios distinct from observed-building claims.

Publish the exact evaluated cases and grading rubric. Require zero known unresolved critical misrepresentation or privacy failures in the demonstration scope. This is a release rule about known issues and tested cases, not a claim of zero unknown risk. Do not advertise all 37 intents as equally reliable merely because each routes. [S1, pp. 2, 7]

### Gate B - supervised operational pilot

Pass Gate A, then demonstrate current permissions, live freshness and quality checks, supported-load behavior, timeout handling, monitored failures, data-retention rules and a tested rollback path. Define who responds to a broken mapping, stale sensor or contested recommendation. Keep the supported scope read-only unless a separately reviewed action capability is explicitly introduced.

Set workload-specific targets for correctness, successful completion and latency before the pilot. This review does not supply invented universal percentages: the appropriate targets depend on the decisions users will make, the tolerance for abstention and the observed workload. Report pilot limitations directly to users.

### Gate C - portability claim

Pass the relevant correctness and policy tests on a second building using the same application revision and declared adapter set. Record all interventions. Distinguish metadata/configuration work from application or prompt changes. A claim of self-service onboarding additionally needs evidence that a person other than the builder can complete the documented process.

## Metrics with useful denominators

| Metric                         | Proposed definition                                                                                 |
|--------------------------------|-----------------------------------------------------------------------------------------------------|
| Substantive-answer correctness | Outputs whose substantive claims all pass the rubric / all substantive outputs.                     |
| Answerable-request completion  | Correct full completions / supported, authorized and answerable requests.                           |
| Unnecessary abstention         | Deliberate abstentions on answerable cases / supported, authorized and answerable requests.         |
| Wrong or unsupported assertion | Count and severity of incorrect or unsupported claims, with claim- and request-level denominators.  |
| Operational outcome            | First-pass success, timeout, error and retry recovery, recorded as separate outcomes.               |
| Evidence completeness          | Published claims containing all required evidence fields / published claims requiring those fields. |

Label answerability independently using the authorized data available at request time. Count correct refusals on unanswerable or restricted cases separately; a timeout is not an abstention. Use N/A for a zero denominator. Grade partial outputs at both claim and request level. Complete metadata alone does not establish correctness.

## ILLUSTRATIVE OUTPUTS, NOT RESULTS FROM THE REAL BUILDING

# Worked examples: better answer behavior

The examples below show how the proposed contracts would change user-facing behavior. Room labels, values and dates are invented fixtures unless explicitly attributed to the source. They are not measurements of OntoSage or recommendations about the actual building.

## Example 1 - a broken reference is not sensor absence

**Situation.** The authorized graph declares a CO2 sensor for Room A, but its required store reference is missing. This mirrors the connection-failure class described in the source, not a new observation of the system. [S1, p. 3]

**Misleading answer:** "Room A has no CO2 sensor. Install one."

**Better answer:** "The building metadata declares a CO2 sensor for Room A, but I could not retrieve its readings because the data connection is incomplete. I cannot assess CO2 conditions for the requested interval."

An operator-visible diagnostic can identify the missing reference. A user without permission to see the sensor inventory may need a coarser message. The same internal distinction supports both truthful diagnosis and appropriate disclosure.

## Example 2 - the heading and the query must describe the same day

**Situation.** On 8 September 2026, a UTC-configured fixture asks for yesterday's maximum. Coverage means nonempty one-hour bins, not continuous monitoring.

**Better answer:** "For 7 September 2026 [00:00, 24:00) UTC, Room A's highest accepted sample was 24.1 degrees C. Samples occur in 22 of 24 hourly bins. The true daily peak is not established; higher values may occur between samples or in empty bins."

The value and bin counts are invented. Nonempty bins do not mean complete hourly monitoring or establish sensor cadence. Record the actual timestamps, bin rule, gaps and rejected samples. Requested dates remain unchanged despite partial coverage; this addresses the source's wrong-day failure class. [S1, pp. 5-6]

## Example 3 - counts are not display limits

**Situation.** A fixture contains 120 matching sensors, but the interface displays at most 50 rows.

**Better answer:** "There are 120 matching sensors in the authorized graph scope. The table shows the first 50; the remaining 70 are not displayed."

The total must come from a valid count or a complete known result set, not the number of rows returned under a limit. If the true total cannot be established, say "50 retrieved" rather than "50 total." The source reports this class of count error. [S1, p. 5]

## Example 4 - mixed evidence should remain visibly mixed

**Situation.** A user asks which rooms exceeded a chosen analytical band and what action the operating manual recommends.

**Better behavior:** Give the observation-based finding with its interval and comparison definition. Separately quote or summarize the applicable procedure with its document revision and location. If the manual is unavailable or conflicts with another authority, state that limitation instead of letting the data lane's success imply that the procedure was answered.

No environmental threshold or operational instruction is supplied here. The example demonstrates source separation and bounded claims, not domain-specific safety guidance.

HOW A TRANSPARENT RANKING SHOULD EXPLAIN ITS LIMITS

# Worked example: an ARBITER dossier

This is a hypothetical fixture, not real data. Preference scores are invented arithmetic inputs, not environmental measurements or standards bands. Provenance labels refer to hypothetical inputs; scores computed from measured inputs are still derived values, not direct measurements.

## Request and admissibility

Suppose the declared task is to rank authorized rooms by two equally weighted preferences for a recorded observation interval. Both criteria require sufficiently complete measured evidence. Availability and future conditions are outside this fixture’s scope. The normalized interpretation and these limits would appear in the dossier.

| Candidate | Thermal preference | Acoustic preference | Provenance / coverage                   | Eligible?               |
|-----------|--------------------|---------------------|-----------------------------------------|-------------------------|
| Room A    | 80                 | 50                  | Measured inputs; coverage passes.       | Yes                     |
| Room B    | 65                 | 75                  | Measured inputs; coverage passes.       | Yes                     |
| Room C    | 95                 | 95                  | Simulation-origin inputs.               | No, in operational mode |
| Room D    | 90                 | Missing             | Required acoustic evidence unavailable. | No, under this policy   |

## Calculation and defensible conclusion

With equal weights,  $\text{score} = 0.5 \times \text{thermal} + 0.5 \times \text{acoustic}$ . Room A scores **65.0** and Room B scores **70.0**. Rooms C and D are excluded before scoring for the operational recommendation. The arithmetic is fully determined by the stated fixture and rule.

A defensible answer is: “Room B ranks first among the two eligible rooms under equal weighting of the two specified preferences. Room C was excluded because its evidence is simulated. Room D lacks a required criterion. This ranking describes the recorded interval; it does not establish future conditions or room availability.”

That statement is narrower than “Room B is the best room.” It also preserves the four-candidate coverage ledger instead of silently showing only the two rooms that happened to be scoreable.

## Sensitivity matters

Change the thermal weight to 0.8 and the acoustic weight to 0.2. Room A becomes **74.0**; Room B becomes **67.0**. The ordering reverses. The conclusion therefore depends on the preference weights, even though both calculations are deterministic.

The practical lesson is not that weighted scoring is wrong. It is that the dossier should distinguish **facts about observations, eligibility policy, preference choices and the resulting recommendation**. A preference-sensitive winner should not be described as universally superior.

## Acceptance checks for this fixture

Under the proposed fixture policy, exclude C even after score transformation. Missing acoustic evidence must not favor D. Verify the stated weight-dependent ranking and disclose authorized exclusions and time limits. Simulation mode may admit C only for an explicitly labeled scenario; it must not become an observation claim.

This example extends the source’s candidate-by-criterion dossier concept without claiming that these fields, weights or behaviors already exist in ARBITER. [S1, p. 4, Fig. 3]

# Proposed request contract

The source describes a checked CQ-IR but supplies no full schema. This is a **proposed, server-assembled request envelope**, not OntoSage's actual interface or executable implementation. Reuse existing fields where appropriate. [S1, p. 4]

```
{
  "schema_version": "proposal-1.1",
  "request_id": "fixture-request-001",
  "original_text": "Maximum temperature in Room A yesterday",
  "building_id": "fixture-building",
  "authorized_scope_ref": "server-resolved-scope-001",
  "space_ids": ["fixture-room-a"],
  "quantity": "air_temperature",
  "operation": "observed_maximum", "output_unit": "degC",
  "time": {
    "reference_time": "2026-09-08T12:00:00Z",
    "building_timezone": "Etc/UTC",
    "start_inclusive": "2026-09-07T00:00:00Z",
    "end_exclusive": "2026-09-08T00:00:00Z"
  },
  "evidence_policy_ref": "measured-lineage-v1",
  "quality_policy_ref": "hour-bin-coverage-v1",
  "defaults_applied": ["building_timezone", "output_unit",
    "evidence_policy_ref", "quality_policy_ref"],
  "assumptions": [
    "Report a sample maximum, not a continuous-day peak."
  ],
  "unresolved_ambiguities": []
}
```

## Invariants to enforce

**Authority.** The server derives scope and allowed policy references; the model cannot grant access or choose a less restrictive policy. Resolve room labels to authorized graph identifiers. Configuration supplies the time zone; product policy supplies unit and evidence defaults.

**Meaning and defaults.** Expose applied defaults and the observed-maximum interpretation. Admit only acceptable assumptions; consequential ambiguity requires clarification or a narrower answer. For averages, add the weighting basis and missing-data policy. The example does not claim a continuous-day peak.

**Evidence.** measured-lineage-v1 is a proposed policy permitting approved transforms of measured samples, not estimates or simulations by default. hour-bin-coverage-v1 records nonempty hourly bins and allows a labeled partial sample summary; it does not imply continuous coverage. Both names are illustrative.

**Limit.** A request object helps prevent reinterpretation only when every consumer honors it. Structural validity does not prove user-intent correctness. Store the interpretation and policy versions with evidence; derive headings and queries from them, and revalidate inherited fields after a follow-up.

MAKE PUBLICATION A DECISION, NOT AN ANNOTATION

# Proposed lane and evidence contracts

The note already describes a shared state bus and one evidence record per answer. This proposal strengthens those existing concepts rather than requiring a separate reasoning framework. Field names below are illustrative. [S1, pp. 3-5]

## Lane result envelope

| Field group         | Proposed contents and purpose                                                                                                             |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Identity and status | Request ID, lane ID, schema version and explicit success, partial, unsupported, restricted or failed status.                              |
| Structured answer   | Typed values, units, entities and operation; requested interval, observed timestamps, coverage segments or bins, and the coverage method. |
| Evidence references | Sensor and store identifiers, retrieval record, graph revision, quality/provenance labels and transformation references.                  |
| Limitations         | Missing criteria, incomplete coverage, rejected rows, unresolved dependencies and acceptable assumptions.                                 |
| Failure details     | Machine-readable reason and protected operator diagnostic; user-facing wording is policy-controlled.                                      |

## Claim-level evidence record

A proposed claim record should identify the exact assertion, its required inputs, transformation, scope, time basis, units, admissible evidence kind, coverage requirement and verification outcome. A comparison claim also depends on candidate eligibility and comparison definitions. A “best” claim depends on the full ranking contract, not only the top candidate’s final score.

Record graph, query-builder, calculation, model/prompt and policy versions where relevant. For mutable data, retain the bounded inputs needed for replay or a durable reference to a recoverable snapshot. A query hash alone is not enough to reproduce a result if the underlying rows later change.

```
For each proposed claim:
  verify the authorized scope and disclosure policy
  verify consistency with the normalized request
  verify admissible provenance and required quality
  verify the calculation and its evidence dependencies
  mark the claim publishable only when all required checks pass

Build the response from publishable claims.
State permitted limitations and unsupported parts explicitly.
Recheck the combined narrative, tables, charts and export.
Publish only if that complete response satisfies the policy.
```

This is design pseudocode, not an implemented verifier. The exact checks must be defined per claim type and tested with intentionally incorrect outputs.

## Why the final recheck matters

Removing one unsupported value can leave a misleading conclusion behind. For example, retaining “best overall” after removing a required criterion still overstates the evidence. A correct table can also be paired with an incorrect heading. The publication decision therefore applies both to individual claims and to their combined meaning.

No verifier described here is assumed infallible. Prefer exact structured checks where available and use test cases to expose the limits of any model-based judgment. The goal is to make known contract violations observable and enforceable, not to claim that another guard eliminates every possible semantic error.

## ADDITIONAL RECOMMENDATIONS, NOT DIAGNOSED VULNERABILITIES

## Operational controls to inspect before a pilot

The source describes a policy engine, privacy refusals, coarsening and exception wrappers, but it does not provide a deployment security model, retention policy or concurrency test. The following is an inspection checklist for a pilot. It should not be read as evidence that these controls are missing. [S1, pp. 2-3, 5]

### Keep the first release read-only

Give retrieval paths only the permissions required for the supported task. Where generated queries or Python exist, determine what they can execute, which resources they can access and what bounds their runtime and result size. Prefer validated builders for routine supported operations. Treat model and document text as data, not as authority to change permissions or invoke new capabilities.

An ECA rule firing in a test should mean that its expected condition/action representation was selected; it need not actuate building equipment. Any physical action capability requires a separately defined permission, approval, failure and recovery design. The supplied note does not establish such an action boundary, so it is outside this review's release recommendation.

### Make operational state visible

A trace should connect the request, interpreted scope, route, graph revision, backend retrieval, calculation, policy checks and publication decision. Record stage latency and first-pass failure separately from retries. A timeout should not later overwrite a completed response with an unreviewed stale result. Cancellation and late-arrival behavior need a defined owner.

Define freshness by quantity and use case, not by copying the source's approximately 30-second row interval across every modality. Display the actual last observation and relevant coverage. A system can be running while a particular series is stale. [S1, p. 3, Fig. 2]

### Treat cache and versioning as part of correctness

If an answer cache exists, key or revalidate results against authorization, request, mapping revision, evidence origin, freshness requirements and policy version. Do not reuse a response across changed permissions merely because its question matches. The source does not establish that such a cache exists; apply this check only where relevant.

Record how graph and readings are aligned for a request. The note describes multiple stores but does not establish a common transactional snapshot. Do not claim cross-store simultaneity without evidence; instead record the retrieval times and consistency policy, or enforce the necessary snapshot mechanism for the task. [S1, p. 3]

### Protect evidence while keeping replay possible

Restrict logs and sensitive evidence; exclude credentials and define retention responsibilities. Coarsening alone does not prove anonymity: evaluate the disclosure policy. User reports need not expose operator diagnostics. These are proposed checks, not diagnosed vulnerabilities. [S1, p. 5]

**Pilot exit evidence** should include permission tests, a backend-outage exercise, a stale-series case, a cache-scope test, a representative load run and a rollback demonstration. The workload and response targets should be chosen for the actual pilot, not inferred from the note's single "typical 14 seconds" figure. [S1, p. 7]

## REQUEST THESE ARTIFACTS BEFORE STRONGER RELIABILITY CLAIMS

# Questions that must be closed

These questions identify gaps in the supplied evidence, not presumed implementation defects. They are deliberately phrased so a repository inspection, trace, policy document or controlled run can answer them.

| Question                                                                 | Why the answer matters                                                                                      | Evidence to request                                               |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Where does an LLM run in each lane?                                      | Resolves the conflict between “classification only” and generated SQL/Python/narration. [S1, pp. 1-2, 6, 8] | Call-site inventory with outputs, validators and execution paths. |
| What is the exact current CQ-IR?                                         | Establishes whether operation, time, units, provenance and unresolved ambiguity are represented. [S1, p. 4] | Schema, validators and accepted/rejected examples.                |
| Which component resolves relative dates?                                 | Prevents query, report and follow-up stages choosing different intervals. [S1, pp. 5-6]                     | A recorded request trace with a frozen-clock regression.          |
| What does grounded=False do today?                                       | Determines whether the reported publication bypass has been closed. [S1, p. 5]                              | Forced-failure test through final rendering and export.           |
| Can simulated data enter operational answers?                            | Visibility alone does not establish admissibility enforcement. [S1, p. 4]                                   | Provenance schema, default policy and a mixed-data test.          |
| Which standards, versions and score transformations does ARBITER use?    | A deterministic ranking still depends on those definitions. [S1, p. 4]                                      | Criterion registry, weighting policy and worked dossier.          |
| How are missing criteria, ties and future periods handled?               | Defines the boundary of “best conditions” claims. [S1, pp. 4, 7]                                            | Eligibility rules, time policy and regression cases.              |
| What are the exact seven earlier failures and three silent record types? | The backlog names the counts but does not enumerate them. [S1, p. 8]                                        | Issue list, reproducible cases and intended answer contracts.     |
| How were the 50 live and 10 demo questions graded?                       | Determines what the reported success actually measures. [S1, p. 7]                                          | Full questions, outputs, rubric, versions and retry records.      |
| Which predicates caused the ECA mismatch?                                | Repair must target the real ontology/query contract, not a guessed property name. [S1, p. 6]                | Rule/query implementation and positive/negative fixtures.         |
| What proves permissions across retrieval, derived results and cache?     | A final refusal alone does not document the complete disclosure boundary. [S1, p. 5]                        | Role-specific end-to-end policy tests.                            |
| What happened when a second building was onboarded?                      | Distinguishes an architectural goal from demonstrated portability. [S1, pp. 3, 8]                           | Same-revision onboarding log with every intervention.             |

## Decision rule

Classify each answer as confirmed, contradicted or unresolved. Update the architecture note to match confirmed behavior. For unresolved questions affecting a release gate, either obtain the evidence or restrict the affected capability. Do not resolve contradictions by silently assuming that the most reassuring passage is the current implementation.

This is the most efficient next review: inspect these specific boundaries rather than requesting an unfocused rewrite or another large count of tests. Each question leads to a concrete artifact and a finite acceptance decision.

## A READY-TO-USE EXECUTION BACKLOG

# The next ten work tickets

The tickets below are a proposed consolidation, not ten additional features. Reuse and close existing source items where they already cover the work. Owners are roles; one builder can hold several. Dependencies indicate safe ordering, not a fixed calendar estimate.

| Ticket                                   | Deliverable / accountable role                                                                                                         | Completion evidence                                                                  |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>T01 - establish baseline</b>          | Capability matrix, revision snapshot, LLM inventory and intent manifest. System owner + engineer.                                      | Supported scope and every execution/response path are named.                         |
| <b>T02 - lock in incidents</b>           | Exact historical failure set and minimal fixtures. Evaluator + engineer. Starts with T01.                                              | Regression detects the historical defect or safe mutation; repaired version passes.  |
| <b>T03 - enforce publication</b>         | Claim-level verification and authorized final response policy. Engineer + privacy owner. After T01-T02.                                | B04, B05, B11 and applicable B12 pass through text, chart and export.                |
| <b>T04 - make data failures explicit</b> | Typed retrieval outcomes, reference validation and capability readiness. Data owner + engineer. After T01-T02.                         | A03-A06 and adapter checks pass; no failure becomes physical absence.                |
| <b>T05 - preserve semantics</b>          | Normalized request, time resolver, count and aggregation contracts. Engineer. After T01-T02.                                           | A07-A14 pass with matching final-output assertions.                                  |
| <b>T06 - govern simulation</b>           | Provenance labels, operational admissibility and explicit simulation mode. Data owner + system owner. After T01.                       | B06 passes; measured coverage is not inflated by simulated Points.                   |
| <b>T07 - finish routing contracts</b>    | Explicit rule precedence, answer collection, in-scope record types and bounded mixed-source behavior. Engineer. After T02, with T05.   | B01, B02, B13 and B14 pass for the supported scope.                                  |
| <b>T08 - prove ARBITER</b>               | Criterion registry, eligibility/coverage rules and reproducible dossiers. System owner + evaluator. After T04-T06.                     | B07-B10 and the worked ranking fixture pass; standards references are explicit.      |
| <b>T09 - evaluate and release</b>        | Held-out results, repeated runs, limitation register and Gate A/B decision. Evaluator + release owner. After relevant earlier tickets. | All 32 cases have results or justified exclusions; first-pass outcomes stay visible. |
| <b>T10 - test portability</b>            | Second-building readiness and intervention log. Data owner + engineer. After a stable evaluated baseline.                              | Gate C evidence exists before generalized scaffolding is prioritized.                |

## First working session

Select the release target, freeze the current version, obtain the source's exact failure list and inspect the final publication path. Reproduce one wrong-answer incident end to end. That provides a real baseline and exposes whether the immediate problem is interpretation, retrieval, policy or rendering. Then size the remaining work from code evidence rather than the orientation note's estimate.

## Final recommendation

Keep the graph-readings separation, ARBITER's typed interpretation and dossier, and the shared evidence record. Verify their contracts, then strengthen confirmed gaps. Historical hardening work is not a reason to replace the design; selected passing tests alone do not establish broader reliability.

The most valuable near-term result is a smaller, clearly supported set of questions with trustworthy answers and explicit inability states. Extend the scope only when the same evidence discipline survives the next lane, dataset and building.

REFERENCES AND LIMITS OF THIS REVIEW

# Sources, source map and terminology

## Primary source

[S1] **How OntoSage Thinks**. Orientation note dated 8 September 2026; nine-page PDF supplied by the user. All system-specific inventory, architecture, incident, scorecard and backlog statements in this report are attributed to this document. Page citations refer to its original PDF pages, not this report’s pagination. No repository, deployment or test result was independently inspected.

| Source location   | What it supports                                                                                 |
|-------------------|--------------------------------------------------------------------------------------------------|
| Page 1            | Reported inventory; opening completion claim; statement about the model’s role.                  |
| Page 2 / Figure 1 | Follow-up rewriting, intent counts, routing rules, document fallback and room-list example.      |
| Page 3 / Figure 2 | Graph-time-series connection, reference failure behavior and orchestrator description.           |
| Page 4 / Figure 3 | Response-collection incident; ARBITER, CQ-IR, dossier, clarification budget and SATURATE.        |
| Page 5 / Figure 4 | Referent/policy/grounding/verifier gates; evidence record; publication, date and count failures. |
| Page 6 / Figure 5 | Specific integration defects; testing claim; broader model role; distinct project goals.         |
| Page 7            | Reported scorecard and claims of demonstrable capability.                                        |
| Page 8            | Nine original backlog items, estimates, proposed ordering and progress measure.                  |
| Page 9            | Retry qualification for earlier live-probe failures.                                             |

## Narrow external implementation references

- [E1] **Brick Ontology Documentation** - [Timeseries Storage](#). Checked 9 September 2026. Supports the Point-to-TimeseriesReference integration pattern. It does not verify OntoSage’s TTL files or storage adapters.
- [E2] **W3C** - [Shapes Constraint Language \(SHACL\)](#). Checked 9 September 2026. Supports graph validation using conditions such as datatype and cardinality constraints. Cited as an implementation option, not as evidence that OntoSage already conforms to a shape set.
- [E3] **pytest documentation** - [About fixtures](#). Checked 9 September 2026. Supports using a known environment or dataset as a test fixture. It does not establish the coverage of the reported OntoSage suite.

## Terms used in this review

**CQ-IR**: the source’s typed, closed-vocabulary representation of a question; its exact schema is not supplied. **Lane**: a routed execution path. **Dossier**: ARBITER’s recorded interpretation, assumptions, coverage and candidate evidence. **Provenance**: the origin and transformation history of evidence, including measured versus simulated status. **Contract**: explicit inputs, outputs and invariants that can be checked. **Abstention**: a deliberate decision not to assert an unsupported answer. **Readiness**: capability-specific evidence that the configured building can currently support a task.

**Review status**: architecture assessment and proposed action plan, based on the supplied note and the three limited references above. No benchmark, code audit, operational approval or environmental-compliance finding is implied.

# Correction log: source fidelity

This log records corrections to review edition 1.0. Source claims remain attributed to the original note; technical refinements are review proposals, not assertions about the running implementation.

## C01 Historical incidents are not a current bug list | pp. 3, 17-18, 29

The earlier review could be read as treating documented incidents as still open. It now distinguishes historical reports, the note's zero-open-P1 scorecard, and unverified present fix status. Work begins by checking existing repairs and regressions, not recreating them. Basis: [S1, pp. 5-7].

## C02 The source estimates do not reconcile | p. 16

The original review missed an arithmetic discrepancy in the orientation note. The six selected estimates sum to 7 days; the three deferred items sum to 9; the visible total is 16, while the source says 19. Keeping the approximately 0.5-day environment-example task would make the selected total about 7.5 if it is additional. The source gives no reconciliation. No new delivery promise is substituted. Basis: [S1, p. 8].

## C03 A hardcoding-check method was not documented | p. 15

The source reports zero hardcoded literals but does not describe a check or its scope. The inferred claim that a particular check found none has been removed. The unresolved contrast with building-specific routing vocabulary and prompt examples remains explicit. Basis: [S1, pp. 7-8].

## C04 Two stores are needed only for relevant query types | pp. 4, 7

The source's simplified join is retained as a source figure, not promoted into a rule for all questions. Graph-only room inventory does not require time-series retrieval. Graph declarations also do not independently prove physical installation. Brick's actual Point-to-reference pattern remains correctly qualified. Basis: [S1, pp. 2-3]; [E1].

## C05 Do not infer absent schemas or a broader model boundary | pp. 9, 17

The shared-state description is not evidence that a lane schema is missing. Conflicting descriptions of model use are not proof that the runtime uses every listed generative capability. Both are now inspection questions with conditional remediation. Basis: [S1, pp. 1-4, 6, 8].

## C06 Shared evidence record and ARBITER dossier are distinct | pp. 22, 29

The source places a shared evidence record after the lanes and a dossier within ARBITER. The conclusion and release requirements now preserve that terminology rather than calling the shared object a central dossier. Basis: [S1, pp. 4-5].

# Correction log: technical precision

## C07 Separate origin, transformation and claim type | pp. 11-12, 21, 24-25

A score or maximum computed from measured inputs is derived, not a raw measurement. The proposal now allows approved measurement-lineage summaries while retaining estimated or simulated ancestry through transformations. Track provenance at observation or batch level, including mixed origins; a Point-level label alone is not enough. The ranking example labels input lineage, not scores as measured.

## C08 Empty rows do not establish whether a series exists | pp. 7-8, 20

The former acceptance wording assumed that every adapter could distinguish an invalid identifier from an empty time window. The revision requires separate series-resolution evidence or an explicit unknown state when that distinction cannot be established. A05/A06 now test that limitation rather than demanding an unsupported diagnosis.

## C09 Coverage is not elapsed time between readings | pp. 6, 23, 26

Requested intervals, observed timestamps and covered bins or segments are now separate. The worked example defines 22 of 24 nonempty hourly bins; it no longer implies 22 hours of continuous monitoring. A sample maximum does not establish the continuous-day peak, even within populated bins.

## C10 Evaluation denominators and abstentions are explicit | pp. 15, 22

Completion is measured on supported, authorized, answerable requests. Correct refusals on restricted or unanswerable cases are recorded separately. Timeouts and errors are not deliberate abstentions. Zero denominators yield N/A. Evidence-field completeness remains distinct from answer correctness.

## C11 Request defaults and policy authority are visible | p. 25

The illustrative JSON records applied defaults and the sample-maximum assumption instead of using an unexplained `measured_only` label and an empty assumption list. The envelope is server-assembled; model text cannot set permissions or bypass policy. The JSON was syntax-checked, not integrated into OntoSage.

## C12 Avoid implying that contracts or guards prove meaning | pp. 6, 12-13, 25

Typed requests are helpful, not universally necessary. A shared object only prevents downstream drift if its consumers honor it. A failed verifier is not proof that the calculation is false. Historical defects are testable, but the note does not reveal today's regression coverage. Mutation checks belong in isolated test environments.

## C13 Conditional features stay conditional | pp. 17, 21, 27

Cache tests apply only if caching exists; generated-execution tests apply to confirmed execution paths. Privacy coarsening is not treated as an anonymity guarantee. These remain proposed inspection criteria, not newly diagnosed implementation vulnerabilities.

# Audit checks and remaining boundaries

This revision checks the 30-page review against all nine pages and figures of *How OntoSage Thinks*. It preserves the six source-aligned chapters, nine original backlog items, 32 proposed regression IDs and ten work tickets. The three external implementation references on page 30 were reopened and checked against their cited uses.

## Editorial checks versus implementation tests

Source attributions, terminology, table arithmetic, worked ranking calculations, example JSON syntax, test-ID coverage, page references and external links were checked. The corrected PDF was rendered for layout inspection. None of these checks is an execution of OntoSage's regression suite, a database audit, or a live-system validation.

## All 32 proposed tests have an accountable work item

| Primary ticket                    | Cases / scope                                                              |
|-----------------------------------|----------------------------------------------------------------------------|
| T04 - data failures and readiness | A01-A06, A15-A16. Data identity, adapter behavior and version consistency. |
| T05 - request semantics           | A07-A14. Dates, counts, operations, units, quality and referents.          |
| T07 - routing and collection      | B01-B03, B13-B14. Includes ECA matching without physical actuation.        |
| T03 - publication and permissions | B04-B05, B11-B12. B12 applies only where a cache exists.                   |
| T06 - simulation governance       | B06. Include transformed data and measured-lineage summaries.              |
| T08 - ARBITER                     | B07-B10. Coverage, weights, future periods and clarification.              |
| T09 - release validation          | B15-B16. Fault handling; generated-execution checks only where applicable. |

T01 establishes scope; T02 supplies incident reproducers across these tickets; T10 tests portability. T09 consolidates results and records a reason for every excluded case. These assignments refine acceptance ownership, not the number of features or a delivery estimate.

## Still requires repository or runtime evidence

The exact LLM call sites, current incident closures, CQ-IR schema, predicate mismatch, seven historical cases, three silent record types, provenance enforcement, standards registry, probe rubric and second-building results remain unverified. Obtain the artifacts listed on page 28; do not infer that any undocumented control is absent.